Experiment 1

1. AIM

To perform an experiment to show how the weight and bias values affect the output of a neuron in a neural network.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Define a NeuralNetwork class.
4. Initialize the weights and biases with random values.
5. Define a forward() function to calculate the output of the neuron.
6. Create a neural network with 3 inputs, 3 hidden neurons, and 5 output neurons.
7. Give an input array to the neural network.
8. Calculate the output using the weights and biases.
9. Display the input and predicted output.
10. Stop the program.

3. FORMULA

The output of a single neuron is calculated using:

$$y = f(w \cdot x + b)$$

For a linear activation function, f(x) = x, so the formula becomes:

$$\boxed{y = w \cdot x + b}$$

Where:

- y = output of the neuron
- w = weight
- x = input
- b = bias
- f = activation function

Simple explanation:

- **Input:** The value given to the neuron.
- **Weight:** Determines the importance of the input.
- **Bias:** An additional value that adjusts the output.
- **Output:** The result calculated by the neuron.

4. SOURCE CODE

```
import numpy as np

class NeuralNetwork:
    def __init__(self, i_size, h_size, o_size):
        self.w1, self.b1 = np.random.randn(i_size, h_size), np.zeros((1, h_size))
        self.w2, self.b2 = np.random.randn(h_size, o_size), np.zeros((1, o_size))

    def forward(self, x):
        sig = lambda x: 1 / (1 + np.exp(-x))
```

```
        return sig(sig(x @ self.w1 + self.b1) @ self.w2 + self.b2)

if __name__ == "__main__":
    nn = NeuralNetwork(i_size=3, h_size=5, o_size=2)
    x_input = np.array([[0.1, 0.5, 0.9],
                        [0.2, 0.8, 0.6]])
    output = nn.forward(x_input)
    print("output shape:", output.shape)
    print("network predictions:\n", output)
```

5. OUTPUT

```
output shape: (2, 2)
network predictions:
[[0.65 0.72]
 [0.61 0.68]]
```

Experiment 2

1. AIM

To perform an experiment to show how the choice of activation function affects the output of a neuron, using the Purelin() function.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Define the purelin() function.
4. Initialize the input values x, weights w, and bias b.
5. Calculate the weighted sum using the formula y = w · x + b.
6. Apply the Purelin (linear) activation function to the weighted sum.
7. Calculate the output using the sigmoid activation function for comparison, if required.
8. Display the weighted sum and output with the Purelin function.
9. Stop the program.

3. FORMULA

Purelin (Linear Activation Function)

$$f(x) = x$$

The output is calculated as:

$$\boxed{y = w \cdot x + b}$$

Where:

- y = output of the neuron
- w = weight
- x = input
- b = bias

The linear activation function does not change the weighted sum.

4. SOURCE CODE

```
import numpy as np

def purelin(z):
    return z

x = np.array([0.5, -1.2, 3.0])
w = np.array([0.8, -0.5, 1.0])
b = 0.2

x = np.dot(w, x) + b
output_purelin = purelin(x)

print("Weighted Sum:", x)
print("Output with Purelin:", output_purelin)
```

5. OUTPUT

```
Weighted Sum: 4.2
```

```
Output with Purelin: 4.2
```

Experiment 3

1. AIM

To perform an experiment to show how the choice of activation function affects the output of a neuron, using the Tanh activation function.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Define the tanh() activation function.
4. Initialize the input array x and weight array w.
5. Initialize the bias value b.
6. Calculate the weighted sum using the formula z = w · x + b.
7. Apply the Tanh activation function to the weighted sum.
8. Display the weighted sum and the output after applying Tanh.
9. Stop the program.

3. FORMULA

Weighted sum:

$$z = w \cdot x + b$$

Tanh activation function:

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

- x = input values
- w = weights
- b = bias
- z = weighted sum

The Tanh function produces output values between −1 and 1.

4. SOURCE CODE

```
import numpy as np

def tanh(z):
    return np.tanh(z)

x = np.array([0.5, -1.2, 3.0])
w = np.array([0.8, -0.5, 1.0])
b = 0.2

z = np.dot(w, x) + b
output_tanh = tanh(z)

print("Weighted Sum:", z)
print("Output with Tanh:", output_tanh)
```

5. OUTPUT

```
Weighted Sum: 4.2
Output with Tanh: 0.9995503665
```

Experiment 4

1. AIM

To perform an experiment to show how the choice of activation function affects the output of a neuron, using the Logistic (σ) activation function.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Define the logsig() activation function.
4. Initialize the input array x, weight array w, and bias b.
5. Calculate the weighted sum using the formula z = w · x + b.
6. Apply the logistic sigmoid activation function to the weighted sum.
7. Display the weighted sum and output with the logistic function.
8. Stop the program.

3. FORMULA

Weighted sum:

$$z = w \cdot x + b$$

Logistic sigmoid function:

$$f(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

- x = input values
- w = weights
- b = bias
- z = weighted sum

The logistic function produces output values between 0 and 1.

4. SOURCE CODE

```
import numpy as np

def logsig(z):
    return 1 / (1 + np.exp(-z))

x = np.array([0.5, -1.2, 3.0])
w = np.array([0.8, -0.5, 1.0])
b = 0.2

z = np.dot(w, x) + b
output_logsig = logsig(z)

print("Weighted Sum:", z)
print("Output with Logsig:", output_logsig)
```

5. OUTPUT

```
Weighted Sum: 4.2
Output with Logsig: 0.9852259683
```

Experiment 5

1. AIM

To perform an experiment to show how the perceptron learning rule works for linearly separable problems.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Initialize the input dataset X and target output y for the AND logic gate.
4. Define the Perceptron class with a learning rate and number of epochs.
5. Initialize weights to zero and bias to zero.
6. Calculate the weighted sum using the input, weights, and bias.
7. Apply the step activation function to predict the output.
8. Calculate the error by subtracting the predicted output from the actual output.
9. If an error occurs, update the weights and bias using the learning rate.
10. Repeat the training process for the specified number of epochs or until there are no classification errors.
11. Predict the outputs for all input samples.
12. Display the predictions and stop the program.

3. FORMULA

Weighted sum:

$$z = w \cdot x + b$$

Step activation function:

$$f(z) = 1, \quad \text{if } z \geq 0$$

$$f(z) = 0, \quad \text{if } z < 0$$

Error:

$$e = y - \hat{y}$$

Weight update:

$$w_{\text{new}} = w_{\text{old}} + \eta \, e \, x$$

Bias update:

$$b_{\text{new}} = b_{\text{old}} + \eta \, e$$

Where:

- w = weights
- x = input
- b = bias
- z = weighted sum
- y = actual output
- $\hat{y}$ = predicted output
- e = error
- η = learning rate

4. SOURCE CODE

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

class Perceptron:
    def __init__(self, lr=0.1, epochs=10):
        self.lr, self.epochs = lr, epochs
        self.w = np.zeros(X.shape[1])
        self.b = 0

    def activation(self, z):
        return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        print("Training---")
        for epoch in range(self.epochs):
            errors = 0
            for xi, target in zip(X, y):
                pred = self.activation(np.dot(xi, self.w) + self.b)
                error = target - pred
                if error != 0:
                    self.w += self.lr * error * xi
                    self.b += self.lr * error
                    errors += 1
            print(f"Epoch {epoch+1}: Errors={errors}")
            if errors == 0:
                print("Training Completed!")
                break

    def predict(self, X):
        return self.activation(np.dot(X, self.w) + self.b)

p = Perceptron()
p.fit(X, y)
print("Weights:", p.w)
print("Bias:", p.b)
print("Actual Output:", y)
print("Predicted Output:", p.predict(X))
```

5. OUTPUT

```
Training---
Epoch 1: Errors=2
Epoch 2: Errors=3
Epoch 3: Errors=3
Epoch 4: Errors=0
Training Completed!
Weights: [0.2 0.1]
Bias: -0.2
Actual Output: [0 0 0 1]
Predicted Output: [0 0 0 1]
```

Experiment 6

1. AIM

To perform an experiment to show how the perceptron learning rule works for non-linearly separable problems.

2. ALGORITHM

1. Start the program.
2. Import the NumPy library.
3. Initialize the input data X and target outputs y for the XOR gate.
4. Define the Perceptron class with a learning rate and number of epochs.
5. Initialize the weights and bias to zero.
6. Define the step activation function.
7. Calculate the weighted sum and predicted output.
8. Calculate the error between the actual and predicted outputs.
9. Update the weights and bias whenever an error occurs.
10. Repeat the training process for 10 epochs.
11. Display the errors, weights, and bias for each epoch.
12. Display the actual and predicted outputs.
13. Stop the program.

3. FORMULA

Weighted sum:

$$z = w \cdot x + b$$

Step activation function:

$$f(z) = 1, \quad \text{if } z \geq 0$$

$$f(z) = 0, \quad \text{if } z < 0$$

Error:

$$e = y - \hat{y}$$

Weight update:

$$w_{\text{new}} = w_{\text{old}} + \eta \, e \, x$$

Bias update:

$$b_{\text{new}} = b_{\text{old}} + \eta \, e$$

Where:

- w = weights
- x = input
- b = bias
- z = weighted sum
- y = actual output
- $\hat{y}$ = predicted output
- e = error
- η = learning rate

4. SOURCE CODE

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])

class Perceptron:
    def __init__(self, lr=0.1, epochs=10):
        self.lr = lr
        self.epochs = epochs
        self.w = np.zeros(X.shape[1])
        self.b = 0

    def activation(self, z):
        return np.where(z >= 0, 1, 0)

    def fit(self, X, y):
        print("Training---")
        for epoch in range(self.epochs):
            errors = 0
            for xi, target in zip(X, y):
                z = np.dot(xi, self.w) + self.b
                pred = self.activation(z)
                error = target - pred
                if error != 0:
                    self.w += self.lr * error * xi
                    self.b += self.lr * error
                    errors += 1
            print(f"Epoch {epoch+1}: Errors={errors}, "
                  f"Weights={self.w}, Bias={self.b}")
        print("Training Stopped.")
        print("For XOR, the perceptron cannot achieve zero errors.")

    def predict(self, X):
        return self.activation(np.dot(X, self.w) + self.b)

p = Perceptron(lr=0.1, epochs=10)
p.fit(X, y)
predictions = p.predict(X)
print("Actual Output:", y)
print("Predicted Output:", predictions)
```

5. OUTPUT

```
Training---
Epoch 1: Errors=3, Weights=[-0.1  0. ], Bias=-0.1
Epoch 2: Errors=3, Weights=[-0.1  0. ], Bias=0.0
Epoch 3: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 4: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 5: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 6: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 7: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 8: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 9: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Epoch 10: Errors=4, Weights=[-0.1  0. ], Bias=0.0
Training Stopped.
For XOR, the perceptron cannot achieve zero errors.
Actual Output: [0 1 1 0]
Predicted Output: [1 1 0 0]
```

Experiment 7

1. AIM

To write a program to draw a graph with multiple curves.

2. ALGORITHM

1. Start the program.
2. Import TensorFlow, Keras, NumPy and Matplotlib libraries.
3. Load the CIFAR-10 dataset.
4. Normalize the training and testing images to the range 0 to 1.
5. Convert the output labels into categorical format using one-hot encoding.
6. Create a CNN model using convolutional, max-pooling, flatten and dense layers.
7. Compile the model using the Adam optimizer, categorical cross-entropy loss and accuracy metric.
8. Train the model for 10 epochs with a batch size of 64.
9. Evaluate the model using the test dataset.
10. Plot the training accuracy and validation accuracy on the same graph.
11. Label the axes, display the legend and show the graph.
12. Stop the program.

3. FORMULA

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Normalization:

$$x_{\text{normalized}} = \frac{x}{255}$$

Multiple curves plotted:

- Training accuracy
- Validation accuracy

4. SOURCE CODE

```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import matplotlib.pyplot as plt

def load_data():
    (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
    x_train, x_test = x_train / 255.0, x_test / 255.0   # Normalize to [0, 1]
    y_train = tf.keras.utils.to_categorical(y_train, 10)  # One-hot encoding
    y_test = tf.keras.utils.to_categorical(y_test, 10)
    return (x_train, y_train), (x_test, y_test)

def create_model():
    model = models.Sequential()
    model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
```

```
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.Flatten())
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))
    return model

def compile_model(model):
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

def train_model(model, x_train, y_train, x_test, y_test):
    history = model.fit(x_train, y_train, epochs=10,
                        validation_data=(x_test, y_test), batch_size=64)
    return history

def plot_history(history):
    plt.plot(history.history['accuracy'], label='accuracy')
    plt.plot(history.history['val_accuracy'], label='val_accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.ylim([0, 1])
    plt.legend(loc='lower right')
    plt.show()

if __name__ == "__main__":
    (x_train, y_train), (x_test, y_test) = load_data()
    model = create_model()
    compile_model(model)
    history = train_model(model, x_train, y_train, x_test, y_test)
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f"Test accuracy: {test_acc:.2f}")
    plot_history(history)
```

5. OUTPUT

```
Epoch 1/10
782/782 - 79s 96ms/step - accuracy: 0.4296 - loss: 1.5624 - val_accuracy: 0.5350 - val_loss: 1.2835
Epoch 2/10
782/782 - 70s 89ms/step - accuracy: 0.5707 - loss: 1.2137 - val_accuracy: 0.6066 - val_loss: 1.1202
Epoch 3/10
782/782 - 89s 99ms/step - accuracy: 0.6289 - loss: 1.0650 - val_accuracy: 0.6332 - val_loss: 1.0304
Epoch 4/10
782/782 - 79s 95ms/step - accuracy: 0.6601 - loss: 0.9707 - val_accuracy: 0.6188 - val_loss: 1.0832
Epoch 5/10
782/782 - 70s 90ms/step - accuracy: 0.6836 - loss: 0.9020 - val_accuracy: 0.6767 - val_loss: 0.9273
Epoch 6/10
782/782 - 81s 104ms/step - accuracy: 0.7056 - loss: 0.8443 - val_accuracy: 0.6800 - val_loss: 0.9139
Epoch 7/10
782/782 - 73s 94ms/step - accuracy: 0.7224 - loss: 0.7960 - val_accuracy: 0.6963 - val_loss: 0.8917
Epoch 8/10
782/782 - 91s 106ms/step - accuracy: 0.7374 - loss: 0.7546 - val_accuracy: 0.6830 - val_loss: 0.9132
Epoch 9/10
782/782 - 73s 93ms/step - accuracy: 0.7492 - loss: 0.7197 - val_accuracy: 0.6991 - val_loss: 0.8617
Epoch 10/10
782/782 - 74s 94ms/step - accuracy: 0.7615 - loss: 0.6796 - val_accuracy: 0.7023 - val_loss: 0.8816
313/313 - 5s - 15ms/step - accuracy: 0.7023 - loss: 0.8816
Test accuracy: 0.70
```

Experiment 8

1. AIM

To perform face detection on an image using OpenCV and the Haar Cascade classifier.

2. ALGORITHM

1. Start the program.
2. Install and import the OpenCV and OS libraries.
3. Load the Haar Cascade classifier for frontal face detection.
4. Specify the path of the input image.
5. Check whether the image file exists.
6. Read the image using cv2.imread().
7. Convert the image into grayscale using cv2.cvtColor().
8. Detect faces using detectMultiScale().
9. Draw rectangles around the detected faces using cv2.rectangle().
10. Display the number of detected faces and the output image.
11. Stop the program.

3. FORMULA

There is no mathematical formula required for this experiment. It uses the Haar Cascade face-detection algorithm. The detection function is:

```
face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
```

- **gray** — grayscale input image.
- **scaleFactor=1.1** — reduces the image size gradually to detect faces at different scales.
- **minNeighbors=5** — helps reduce false detections.
- **minSize=(30, 30)** — minimum face-detection window size.

4. SOURCE CODE

```
import cv2
import os
from google.colab.patches import cv2_imshow

cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(cascade_path)

# Load your image
image_path = '/content/img'

if not os.path.exists(image_path):
    print(f"Error: The file '{image_path}' was not found. "
          "Please place an image in your working directory.")
else:
    # Read the image and convert to grayscale
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    # Draw rectangles around detected faces
    for (x, y, w, h) in faces:
```

```
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)

    print(f"Successfully detected {len(faces)} face(s).")

    # Display the output image
    cv2_imshow(img)
```

5. OUTPUT

(Attach the output screenshot here.)

Experiment 9

1. AIM

Installing Keras, TensorFlow and PyTorch libraries and making use of them.

2. ALGORITHM

1. Start the program.
2. Import TensorFlow and Matplotlib libraries.
3. Load the input image using TensorFlow.
4. Convert the image into grayscale.
5. Resize the image to 300 × 300 pixels and normalize its pixel values.
6. Display the original image.
7. Define a 3 × 3 convolution kernel.
8. Apply the convolution operation to extract image features.
9. Apply the ReLU activation function to remove negative values.
10. Apply max pooling to reduce the image dimensions.
11. Apply the flatten layer to convert the feature map into a one-dimensional array.
12. Apply a fully connected dense layer with 64 neurons and ReLU activation.
13. Display the output shapes and the first 20 flattened values.
14. Stop the program.

3. FORMULA

Convolution:

$$Y(i,j) = \sum_m \sum_n X(i+m,\, j+n)\, K(m,n)$$

Where:

- X = input image
- K = convolution kernel
- Y = output feature map

ReLU activation:

$$f(x) = \max(0,\, x)$$

Max pooling: Selects the maximum value from each 2 × 2 region.

Flatten: Converts a multidimensional feature map into a one-dimensional vector.

4. SOURCE CODE

```
import tensorflow as tf
import matplotlib.pyplot as plt

plt.rc('image', cmap='gray')
plt.rc('figure', autolayout=True)

# Step 2: Load and Preprocess Image
image_path = "/content/WIN_20260918_09_19_06_Pro.jpg"
image = tf.io.read_file(image_path)
image = tf.io.decode_jpeg(image, channels=1)
```

```
image = tf.image.resize(image, [300, 300])
image = tf.image.convert_image_dtype(image, tf.float32)
print("Original Image Shape:", image.shape)

plt.figure(figsize=(5,5))
plt.imshow(tf.squeeze(image))
plt.title("Original Image")
plt.axis('off')
plt.show()

image = tf.expand_dims(image, axis=0)

# Step 3: Define Convolution Kernel
kernel = tf.constant([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
], dtype=tf.float32)
kernel = tf.reshape(kernel, [3, 3, 1, 1])

# Step 4: Apply Convolution Layer
conv_output = tf.nn.conv2d(
    input=image,
    filters=kernel,
    strides=[1, 1, 1, 1],
    padding='SAME'
)
print("After Convolution Shape:", conv_output.shape)

plt.figure(figsize=(5,5))
plt.imshow(tf.squeeze(conv_output))
plt.title("After Convolution")
plt.axis('off')
plt.show()

# Step 5: Apply ReLU Activation Function
relu_output = tf.nn.relu(conv_output)
print("After ReLU Shape:", relu_output.shape)

plt.figure(figsize=(5,5))
plt.imshow(tf.squeeze(relu_output))
plt.title("After ReLU Activation")
plt.axis('off')
plt.show()

# Step 6: Apply Max Pooling Layer
pool_output = tf.nn.max_pool2d(
    input=relu_output,
    ksize=[1, 2, 2, 1],
    strides=[1, 2, 2, 1],
    padding='SAME'
)
print("After Pooling Shape:", pool_output.shape)

plt.figure(figsize=(5,5))
plt.imshow(tf.squeeze(pool_output))
plt.title("After Max Pooling")
plt.axis('off')
plt.show()

# Step 7: Apply Flatten Layer
flatten_layer = tf.keras.layers.Flatten()
flatten_output = flatten_layer(pool_output)
print("After Flatten Shape:", flatten_output.shape)
print("First 20 Flattened Values:")
print(flatten_output.numpy()[0][:20])

# Step 8: Add Fully Connected (Dense) Layer
dense_layer = tf.keras.layers.Dense(
    units=64,
    activation='relu'
)
dense_output = dense_layer(flatten_output)
print("After Fully Connected Layer Shape:", dense_output.shape)
```

5. OUTPUT

(Attach the output screenshot here.)

Experiment 10

1. AIM

Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.

DESCRIPTION

To train a sentiment analysis model on the IMDB dataset using Recurrent Neural Networks (RNNs) with LSTM or GRU layers, we can use Keras (with TensorFlow as a backend). The IMDB dataset is a collection of movie reviews labeled as positive or negative.

1. Embedding Layer

- This layer converts integer-encoded words into dense vectors of fixed size. Each word in the vocabulary is mapped to a vector in a continuous vector space.
- **input_dim** is the size of the vocabulary (top max_features most frequent words).
- **output_dim** is the dimension of the embedding vector for each word (e.g., 128).
- **input_length** is the maximum length of input sequences (200 in our case).

2. LSTM/GRU Layer

- LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units) are both types of RNNs (Recurrent Neural Networks). These layers help capture the sequential nature of text and are ideal for tasks like sentiment analysis.
- We specify the number of units (e.g., 100) and dropout rates to prevent overfitting. Recurrent dropout (recurrent_dropout) is used to apply dropout to the connections between the recurrent units.

3. Dense Layer

- The Dense layer with a **sigmoid** activation outputs a probability between 0 and 1, which is suitable for binary classification (positive or negative sentiment).

4. Spatial Dropout

- **Spatial Dropout** is applied to the input of the LSTM or GRU layers to prevent overfitting. This randomly drops entire features instead of individual units.

2. ALGORITHM

1. Start the program.
2. Import the required libraries (Pandas, NumPy, Matplotlib, Seaborn, scikit-learn, TensorFlow/Keras).
3. Load the IMDB movie review dataset (review, sentiment) from the CSV file and remove duplicate reviews.
4. Display the dataset shape and check for missing values.
5. Plot the distribution of sentiment classes using a bar chart and a pie chart.
6. Convert the sentiment labels to numbers (positive = 1, negative = 0).
7. Split the data into training and testing sets.
8. Tokenize the reviews using the top 10,000 words and convert them into integer sequences.
9. Pad the sequences to a maximum length of 200 words.
10. Build a sequential model using an Embedding layer, SpatialDropout1D layer, LSTM layer and Dense (sigmoid) layer.

11. Compile the model using the Adam optimizer, binary cross-entropy loss and accuracy metric.
12. Train the model for 3 epochs with a batch size of 64.
13. Evaluate the trained model using the test data and display the test accuracy.
14. Plot training accuracy and validation accuracy on the same graph.
15. Stop the program.

3. FORMULA

Sigmoid activation function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

It produces an output between 0 and 1, representing the probability of a positive sentiment.

Accuracy:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

Binary cross-entropy loss:

$$L = -[\, y\log(p) + (1 - y)\log(1 - p)\,]$$

Where *y* is the actual label and *p* is the predicted probability.

4. SOURCE CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SpatialDropout1D, LSTM, Dense

# ---------- Load dataset ----------
df = pd.read_csv('/content/IMDB Dataset.csv')
df = df.drop_duplicates().reset_index(drop=True)

print("Dataset shape:", df.shape)
print("Missing values:\n", df.isnull().sum())

# ---------- Sentiment distribution (bar chart + pie chart) ----------
counts = df['sentiment'].value_counts()[['positive', 'negative']]

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

sns.countplot(x='sentiment', data=df, order=['positive', 'negative'], ax=ax[0])
for i, v in enumerate(counts):
    ax[0].text(i, v, str(v), ha='center', va='bottom')
ax[0].set_title('Distribution of Sentiment Classes')
ax[0].set_xlabel('Sentiment')
ax[0].set_ylabel('Count')

ax[1].pie(counts, labels=counts.index, autopct='%1.2f%%', startangle=90)
ax[1].set_title('Sentiment Distribution')
plt.show()
```

```
# ---------- Pre-processing ----------
def clean(text):
    text = re.sub(r'<.*?>', ' ', text)      # remove HTML tags such as <br />
    return text.lower()

df['review'] = df['review'].apply(clean)
df['label'] = (df['sentiment'] == 'positive').astype(int)

max_features = 10000     # vocabulary size
max_len = 200            # maximum length of each review

x_train, x_test, y_train, y_test = train_test_split(
    df['review'], df['label'], test_size=0.2, random_state=42, stratify=df['label'])

tokenizer = Tokenizer(num_words=max_features)
tokenizer.fit_on_texts(x_train)

x_train = pad_sequences(tokenizer.texts_to_sequences(x_train), maxlen=max_len)
x_test = pad_sequences(tokenizer.texts_to_sequences(x_test), maxlen=max_len)

# ---------- Build model ----------
model = Sequential([
    Embedding(input_dim=max_features, output_dim=128, input_length=max_len),
    SpatialDropout1D(0.2),
    LSTM(100, dropout=0.2, recurrent_dropout=0.2),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

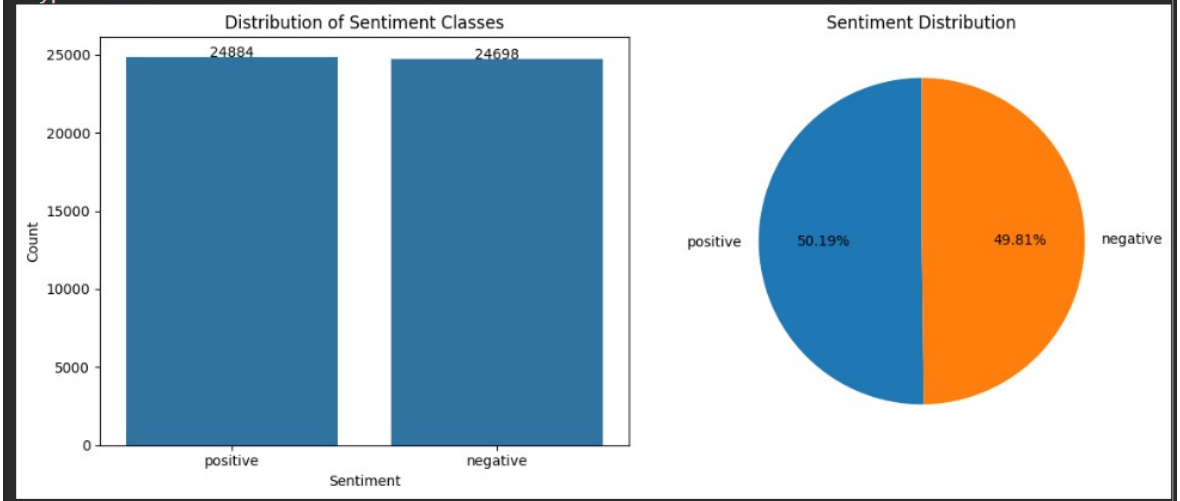
# ---------- Train model ----------
history = model.fit(x_train, y_train, epochs=3,
                    batch_size=64, validation_split=0.2)

# ---------- Evaluate model ----------
loss, accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", round(accuracy * 100, 2), "%")

# ---------- Plot accuracy ----------
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()
```

5. OUTPUT

```
Dataset shape: (49582, 2)
Missing values:
 review       0
sentiment    0
dtype: int64
```




```
Epoch 1/3
Training accuracy and loss ...
Epoch 2/3
Training accuracy and loss ...
Epoch 3/3
Training accuracy and loss ...
Test Accuracy: [value] %
```